

Going Further With Winograd Convolutions: Tap-Wise Quantization for Efficient Inference on 4x4 Tiles

Renzo Andri*, Beatrice Bussolino*^{†1}, Antonio Cipolletta*, Lukas Cavigelli*, Zhe Wang*

*Computing Systems Lab, [Huawei Zurich Research Center](#)

[†]DET, [Politecnico di Torino](#), Italy

Abstract—Most of today’s computer vision pipelines are built around deep neural networks, where convolution operations require most of the generally high compute effort. The Winograd convolution algorithm computes convolutions with fewer multiply–accumulate operations (MACs) compared to the standard algorithm, reducing the operation count by a factor of 2.25× for 3×3 convolutions when using the version with 2×2-sized tiles F_2 . Even though the gain is significant, the Winograd algorithm with larger tile sizes, i.e., F_4 , offers even more potential in improving throughput and energy efficiency, as it reduces the required MACs by 4×. Unfortunately, the Winograd algorithm with larger tile sizes introduces numerical issues that prevent its use on integer domain-specific accelerators (DSAs) and higher computational overhead to transform input and output data between spatial and Winograd domains.

To unlock the full potential of Winograd F_4 , we propose a novel tap-wise quantization method that overcomes the numerical issues of using larger tiles, enabling integer-only inference. Moreover, we present custom hardware units that process the Winograd transformations in a power- and area-efficient way, and we show how to integrate such custom modules in an industrial-grade, programmable DSA. An extensive experimental evaluation on a large set of state-of-the-art computer vision benchmarks reveals that the tap-wise quantization algorithm makes the quantized Winograd F_4 network almost as accurate as the FP32 baseline. The Winograd-enhanced DSA achieves up to 1.85× gain in energy efficiency and up to 1.83× end-to-end speed-up for state-of-the-art segmentation and detection networks.

Keywords—Machine Learning Acceleration, Winograd Convolution, ML System Design

I. INTRODUCTION

Convolutional neural networks (CNNs) have had a significant breakthrough in almost all Artificial Intelligence (AI) tasks in recent years, thanks to large, comprehensible, and publicly available data sets, easy-to-use frameworks, and the newly available vast compute resources. Nevertheless, state-of-the-art neural networks come with high computational costs (>GFLOPs/inference) and memory requirements (>10 MB), which has led to an explosion of DSAs in datacenters [21], [36] and edge devices [22].

Typically, floating-point numbers have been used to flexibly represent CNN computations, but floating-point

datapath is very area- and power-hungry due to the large intermediate values, re-normalization, and exception handling, which require adequate hardware support. Thanks to the intrinsic error tolerance of neural networks [12], [70], 8-bit integer operations can be used for most inference applications with no to minimal accuracy degradation. As integer operations—like additions and multiplications—are one order of magnitude more energy-efficient than their floating-point counterparts [17], integer-based accelerators can achieve much higher peak throughput and energy efficiency.

In order to reduce operation count and memory footprint even further, recent works have proposed to adopt smaller convolutional kernel sizes [27], exploit sparsity [14], use group convolutions [27], [63], channel shuffling [68], and depthwise separable convolutions [18]. Nevertheless, compute-heavy, dense 3×3 convolutional layers are still widely used in many state-of-the-art computer vision models [39], [53], [54]. Thus, the adoption of the Winograd algorithm [62] represents an interesting optimization opportunity as it converts the 3×3 convolution operation into a much less expensive elementwise multiplication.

The Winograd convolution algorithm extends the Toom–Cook algorithm to support convolutions by applying the (polynomial) Chinese remainder theorem, minimizing the number of required multiplications [6], [62]. Specifically, the 2D convolution with a feature map x of size $m \times m$ and convolutions kernel f of size 3×3 is calculated with the Winograd (convolution) algorithm $F_m = F(m \times m, 3 \times 3)$ as follows:

$$Y = A^T [(GfG^T) \odot (B^T xB)] A \quad (1)$$

where $G \in \mathbb{R}^{(m+2) \times 3}$, $B^T \in \mathbb{R}^{(m+2) \times (m+2)}$, and $A^T \in \mathbb{R}^{m \times (m+2)}$ are called transformation matrices. Specifically, the G and B^T matrices transform the weights and the input feature maps, respectively, from the spatial domain to the Winograd domain. Here, the convolution becomes an $(m+2)^2$ -sized element-wise multiplication of the feature maps with the filter weights such that the number of multiplications is reduced from $m^2 \cdot 9$ to $(m+2)^2$. Then, the A^T matrix transforms the output feature maps back to the spatial domain.

While larger feature map tile sizes ($m \uparrow$) reduce the number of required multiply-and-accumulate operations

¹This work was done while Beatrice Bussolino was an intern at Huawei Technologies - Zurich Research Center.